

Kaggle Competition Report

Irrigation Need Prediction

Optuna-Tuned Full Stacking Pipeline

Student Information	Details
Name	<u>Sana Munir Alam</u>
Roll No	<u>24K-0573</u>
Section	<u>BCS-4E</u>
Submission Date	<u>27-04-2026</u>

Final Leaderboard Result: Kaggle public score: 0.97144 | Rank: 766

This report summarizes the final machine learning pipeline, its preprocessing, validation strategy, model experiments, and leaderboard performance. The final solution uses full stacking with LightGBM, XGBoost, and CatBoost as base learners and Logistic Regression as the meta-learner.

Executive Summary

The final solution is a production-ready multiclass classification pipeline for predicting irrigation need as Low, Medium, or High. The pipeline was rebuilt from a fixed-hyperparameter and fixed-weight ensemble approach into a full stacking system with Bayesian hyperparameter optimization using Optuna.

- Final external Kaggle score: 0.97144.
- Final Kaggle rank: 766.
- Primary optimization metric: balanced accuracy, selected because the target classes may be imbalanced.
- Final architecture: Optuna-tuned LightGBM, XGBoost, and CatBoost base models stacked with Logistic Regression.
- Leakage control: base-model meta-features are generated strictly from out-of-fold predicted probabilities.
- Reproducibility: shared StratifiedKFold splits, fixed random seed, saved Optuna studies, and exported model artifacts.

train.csv/test.csv

- > feature engineering
- > fold-specific ColumnTransformer preprocessing
- > Optuna CV tuning for each base model
- > OOF probability meta-features
- > Logistic Regression stacker
- > full-data retraining
- > test probabilities
- > submission.csv

1. Competition Overview

Competition Name	Irrigation Need Prediction / Irrigation Need Classification
Problem Type	Supervised multiclass classification
Target Variable	Irrigation_Need with three ordered labels: Low, Medium, and High
Evaluation Metric	balanced_accuracy_score
Final Kaggle Score	0.97144
Final Rank	766

Brief understanding of the dataset: The dataset contains tabular agricultural, weather, soil, crop, and irrigation-related features. Numeric features include soil pH, soil moisture, organic carbon, electrical conductivity, temperature, humidity, rainfall, sunlight hours, wind speed, field area, and

previous irrigation amount. Categorical features include soil type, crop type, crop growth stage,

season, irrigation type, water source, mulching status, and region.

The goal is to classify each field observation into the correct irrigation need level. The problem is nonlinear because irrigation need depends on interactions between soil moisture, temperature, rainfall, wind, sunlight, crop type, and field conditions. It is also sensitive to class imbalance, so balanced accuracy was used instead of plain accuracy.

2. Data Preprocessing

The final script uses a clean preprocessing pipeline built with scikit-learn ColumnTransformer. Preprocessing is fitted separately inside each CV fold during validation, which prevents validation information from leaking into the training fold.

Preprocessing Step	Implementation in Final Pipeline
Missing values handling	Categorical columns: most frequent imputation. Numeric columns: median imputation.
Encoding techniques	OrdinalEncoder with handle_unknown="use_encoded_value" and unknown_value=-1 for unseen categories.
Feature scaling	Not required for the final tree-based base learners. The meta-learner receives probability features in the 0-1 range, so no extra scaling is necessary there.
Train-validation split	5-fold StratifiedKFold with shuffle=True and random_state=42.
Sample weights	Class-balanced sample weights are computed and can be multiplied with a user-supplied sample_weight column. The weights are passed to all base models and the meta-learner.

Feature Engineering Section

The script creates deterministic, domain-inspired features without using the target variable. These features help the gradient boosting models learn irrigation stress, water availability, and soil condition patterns.

Feature Group	Engineered Features
Weather stress	evapo_proxy, wind_drying, sunshine_stress, heat_index
Moisture stress	moisture_deficit, temp_moisture_stress, moisture_sq
Water availability	water_balance, net_water_avail, rain_moisture
Soil health	soil_health, salinity_stress

Kaggle Competition Report - Irrigation Need Prediction

Field normalization	rainfall_per_ha, prev_irr_per_ha, area_log
Interaction terms	moisture_wind, temp_wind, need_proxy

Leakage Prevention Note: All engineered features are computed only from input columns. During CV, the ColumnTransformer is fitted only on training rows and then applied to validation rows.

3. Models Attempted

Baseline and mandatory models were considered to understand model behavior before moving to the final stacking architecture. The final production script keeps the strongest approach: optimized gradient boosting models combined with a stacker.

Model	Role	Observation
Decision Tree	Baseline supervised model	Useful for interpretability but prone to overfitting and weaker generalization.
Naive Bayes	Mandatory probabilistic baseline	Fast but assumes feature independence, which is unrealistic for soil-weather interactions.
K-Means as classifier	Unsupervised baseline	Cluster assignments do not necessarily align with irrigation need labels.
Logistic Regression	Linear baseline and final meta-learner	Limited as a raw-feature model, strong as a stacker on probability features.
Linear Regression OvR	Experimental linear classifier	Not ideal for categorical multiclass classification, used only as a comparison.
Random Forest	Bagged tree baseline	More robust than a single tree but usually less competitive than tuned gradient boosting here.
LightGBM	Advanced base model	Strong on tabular data, tuned with Optuna.
XGBoost	Advanced base model	Strong regularized boosting model, tuned with Optuna.
CatBoost	Advanced base model	Handles categorical-style patterns well, tuned with Optuna.
Full Stacking	Final selected model	Combines OOF probabilities from LightGBM, XGBoost, and CatBoost using Logistic Regression.

The earlier fixed-weight soft-voting ensemble was removed because manual weights can be unstable and require trial-and-error. The final approach lets the meta-learner learn how to combine the three base models from OOF probability features.

4. Cross Validation Strategy

K-Fold details	5-fold StratifiedKFold, shuffle=True, random_state=42.
Shared split usage	The same fold boundaries are reused for Optuna tuning, base OOF probability generation, and meta-model evaluation.
Primary metric	balanced_accuracy_score for every Optuna objective and CV evaluation.
OOF design	Each training row receives base-model probabilities from models that did not train on that row.
Strict stack CV	Optional strict full-stack CV trains inner OOF features within each outer fold to estimate the full stack without leakage.
Best validation accuracy	Generated by the script in outputs/cv_results.csv. The exact internal CV value was not provided with the prompt, while the external Kaggle score was 0.97144.

LOOCV Observations

Leave-One-Out Cross Validation was not used in the final solution. For Optuna-tuned LightGBM, XGBoost, and CatBoost, LOOCV would require an impractical number of model fits and would not provide enough benefit over stratified K-fold CV. Stratified K-fold was a better choice because it preserves the class distribution in each fold while keeping computation feasible.

Why the CV Strategy Is Safe

1. The fold assignment is created once and saved as cv_folds.csv.
2. A fold-specific ColumnTransformer is fitted only on that fold's training rows.
3. Base models produce OOF probabilities only for validation rows not seen during their training.
4. The Logistic Regression meta-learner is trained on OOF probabilities, not on in-sample base predictions.
5. Only after validation is complete are the base models retrained on the full training dataset.

5. Failed Attempts and Insights

The following attempts were useful for comparison but were not selected as the final submission model. Exact baseline CV values and confusion matrix images should be copied from the generated output files if required by the instructor; they were not included in the prompt.

Model Used	Accuracy Obtained	What Went Wrong?	What Was Improved?
Decision Tree	Exact CV value not supplied	Single trees can overfit and are sensitive to split choices.	Moved to boosted tree ensembles for stronger generalization.
Naive Bayes	Exact CV value not supplied	Feature independence assumption is too strong for irrigation data.	Used only as a lightweight baseline.
K-Means classifier	Exact CV value not supplied	Clusters optimize distance, not class separation.	Kept as an educational baseline, not a final candidate.
Raw Logistic/Linear models	Exact CV value not supplied	Linear boundaries underfit nonlinear soil-weather interactions.	Reused Logistic Regression as a meta-learner on probability features.
Fixed soft-voting ensemble	Exact CV value not supplied	Manual weights are not learned and can be suboptimal.	Replaced with full OOF stacking.

CV Results

model	cv_balanced_accuracy_mean	cv_balanced_accuracy_std	oof_balanced_accuracy	oof_accuracy
lightgbm	0.97061511	0.000854288	0.970615163	0.982573016
xgboost	0.972586881	0.000576304	0.972586934	0.983244444
catboost	0.970080283	0.001099709	0.970080389	0.982122222
stacking_meta_cv_fast	0.972926381	0.000686552	0.97292646	0.981261905
stacking_full_cv_strict	0.972978051	0.000694649	0.972978132	0.981069841

Stacking Classification Report

stacking_classification_report.txt

File Edit View H1 ≡ B I S ↺ ↻

Stacking report source: strict full-stack CV

	precision	recall	f1-score	support
Low	0.99	1.00	0.99	369917
Medium	0.99	0.96	0.97	239074
High	0.82	0.96	0.89	21009
accuracy			0.98	630000
macro avg	0.93	0.97	0.95	630000
weighted avg	0.98	0.98	0.98	630000

Confusion matrix, rows=true, cols=pred:

[	368221	1693	3]
[	5133	229620	4321]
[	0	776	20233]]

6. Final Model Selection

Best model	Full stacking ensemble
Base models	LightGBM, XGBoost, CatBoost
Meta-learner	Logistic Regression
Meta-features	OOF predicted probabilities from each base model; 3 models x 3 classes = 9 meta-features
Optimization method	Optuna Bayesian/TPE search with MedianPruner
Selection metric	Mean balanced accuracy over shared StratifiedKFold CV splits
Final training	After tuning, each base model is retrained on the full training dataset using its best parameters. Test probabilities are stacked and passed to the trained meta-model.
Why selected?	It combines diverse high-performing gradient boosting models while avoiding leakage through strict OOF meta-feature construction.

Hyperparameters Tuned by Optuna

Model	Hyperparameters / Search Space
LightGBM	n_estimators, learning_rate, num_leaves, max_depth, min_child_samples, subsample, colsample_bytree, reg_alpha, reg_lambda, min_split_gain
XGBoost	n_estimators, learning_rate, max_depth, min_child_weight, subsample, colsample_bytree, gamma, reg_alpha, reg_lambda, max_bin
CatBoost	iterations, learning_rate, depth, l2_leaf_reg, random_strength, bagging_temperature, border_count
Meta-learner	Logistic Regression C and max_iter are configurable; the code is structured so the meta-learner can be swapped easily.

Production Artifact Note: The final script saves best_params.json, optuna_best_results.json, cv_results.csv, base_oof_probabilities.npz, test_probabilities.npz, final_stacking_model.joblib, and submission.csv.

7. Leaderboard Performance

Kaggle score	0.97144
Rank	766
Submission file	submission.csv generated by the final stacking pipeline
Result interpretation	The score indicates very strong predictive performance on the public leaderboard for the irrigation need multiclass task.

Kaggle Leaderboard Screenshot Placeholder

The screenshot displays the Kaggle Leaderboard for the 'Predicting Irrigation Need' competition. The user's submission is highlighted with a 'Your Best Entry!' message, indicating a score of 0.97144, which is an improvement over their previous score of 0.96918. The table below shows the top performers on the leaderboard.

Rank	Participant	Score	Rank Change	Time
762	Hồ Tuấn Khôi	0.97151	4	11d
763	Denys Vivdenko	0.97150	2	3d
764	denis kashulin	0.97145	61	8h
765	IEEM26_112034042	0.97144	4	5h
766	Sana Munir Alam	0.97144	4	1m
Your Best Entry! Your most recent submission scored 0.97144, which is an improvement over your previous score of 0.96918. Great job!				
767	Parth Zadeshwariya	0.97143	7	15d
768	CUHKLHT	0.97143	18	10d
769	yusuf emre	0.97141	11	18d
770	Ali Shahi	0.97140	13	6h

8. Conclusion and Learnings

Key Insights

- Balanced accuracy is more appropriate than plain accuracy when class distribution is uneven.
- Feature engineering based on moisture stress, evapotranspiration proxies, water balance, and soil health improved the signal available to models.

Kaggle Competition Report - Irrigation Need Prediction

- Gradient boosting models are well-suited to the nonlinear interactions in agricultural tabular data.
- OOF stacking is safer than training a meta-model on in-sample base predictions because it reduces leakage and overfitting.
- Using the same CV splits across tuning, OOF generation, and meta-learning made the experiment more reproducible and easier to audit.

Challenges Faced

- Avoiding leakage while still training a strong stacker required careful OOF probability generation.
- Optuna tuning for three boosting models is computationally expensive, especially with strict stack CV enabled.
- Manual fixed-weight voting was difficult to justify because the best weight combination can change across folds.
- Maintaining sample weights through base models and the meta-learner was important for class-balanced evaluation.

Future Improvements

- Increase Optuna trial counts and use time-budgeted tuning for each base model.
- Try calibrated probabilities before stacking to improve the meta-features.
- Experiment with alternative meta-learners such as RidgeClassifier, LinearSVC with calibration, or a small gradient boosting stacker.
- Add multi-seed averaging after tuning for more stable test probabilities.
- Use SHAP or permutation importance to explain the most influential engineered features.
- Compare public leaderboard performance with private leaderboard performance to

Final Statement: The final submission used a leak-safe, Optuna-tuned stacking pipeline and achieved a Kaggle score of 0.97144 with rank 766.

check for overfitting to public feedback.

Appendix: Script Outputs and Reproducibility

Output Artifact	Purpose
config.json	Stores runtime configuration such as seed, fold count, target column, and Optuna trial counts.
cv_folds.csv	Stores the exact row-to-fold mapping used across the pipeline.
optuna_trials_<model>.csv	Stores trial-level Optuna results for LightGBM, XGBoost, and CatBoost.
best_params.json	Stores the selected best hyperparameters for each base model.

Kaggle Competition Report - Irrigation Need Prediction

base_oof_probabilities.npz	Stores OOF predicted probabilities for the three base models.
meta_train_oof_features.csv	Stores the 9-column OOF meta-feature matrix used by the <code>stacker</code> .
cv_results.csv	Stores base-model and stacking CV metrics.
stacking_classification_report.txt	Stores the classification report and confusion matrix for the stacking model.
feature_importances.csv	Stores feature importances from models that expose <code>feature_importances_</code> .
final_stacking_model.joblib	Stores the final preprocessor, trained base models, meta-model, class map, and feature names.
submission.csv	Final Kaggle submission file.